Testing Azure Local Performance by Using VMFleet

Agenda

- **Objective:** Step through the process of assessing Azure Local performance by using VMFleet
- **Scope:** Implementing a test environment based on VMFleet 2.0 and MSLab

Introduction to VMFleet

- **VMFleet** is a **storage load generation** and **performance-stress tool** for **hyper-converged Azure Local / Azure Stack HCI**.
- Executes **DiskSpd** workloads across a large fleet of lightweight VMs.
- Used to validate **I/O throughput**, **latency**, **CPU behavior**, and **cluster-wide storage performance** under real-world pressure.
- Especially valuable for **benchmarking S2D performance**, confirming hardware configurations, and validating cluster health.

VMFleet Common Use Cases (Azure Local)

- **Baseline cluster performance** after deployment or hardware upgrades.
- **Stress-testing S2D** to detect weak disks, misconfigured networking, or CPU imbalances.
- **Benchmarking NVMe / SSD storage pools**.
- **Evaluating cluster resiliency behaviors** under load (e.g., CSV redirection, rebalancing).
- **Validating performance expectations** from vendors or sizing guidance.

VMFleet Architecture Overview

- VMFleet uses a **“fleet” of Server Core VMs**, all based on a **shared parent VHD**.
- A **Control Loop** orchestrates work:
  - **control.ps1**: central coordination script
  - **run.ps1**: workload parameters
- Logs and scripts stored in the **Collect** CSV:
  - **Control**, **Flag**, **Result**, **Tools** folders
- **DiskSpd** is executed inside each VM and aggregated at cluster scale.

Core Components of VMFleet

- **Server Core VHD / Gold image**:
  - base OS for fleet VMs.
- **VMFleet PowerShell Module**:
  - provides all orchestration commands.
- **DiskSpd**:
  - low-level I/O generator installed in **Tools** folder.
- **CSV Volumes**:
  - one per node, balancing I/O load.
- **Collect Volume**:
  - central storage for scripts and results.

VMFleet Requirements

- **Azure Local** with S2D enabled.
- **CSV per node** plus a dedicated **Collect** CSV.
- **Hyper-V installed** on the management machine to build VHDs.

VMFleet Implementation Workflow

1. Create Windows Server Core VHD
2. Create VMFleet image
3. Configure VMFleet prerequisites & deploy
4. Run tests + collect and analyze results, then cleanup.

Task 1: Creating Windows Server 2022 Core VHD

- Use `CreateParentDisk.ps1` to automate VHD creation.
- Uses `Convert-WindowsImage.ps1` internally.
- Requires Windows Server ISO (with **install.wim**).
- Size **~30GB** to reduce storage usage when converted to fixed VHD.

Task 2: Creating the VMFleet Image

- Use **CreateVMFleetDisk.ps1** to generate VMFleet-optimized VHD
- Script prompts for:
  - Base VHD
  - VM admin password

Task 3: Configuring VMFleet Prerequisites

- Installing PowerShell modules:
  - NuGet Provider, VMFleet, PrivateCloud.DiagnosticInfo
- Creating volumes across nodes:
  - 1 CSV per node
  - Collect volume (100–200GB depending on environment)
- Copying VMFleet module and VHD to each cluster node.

Task 3: Setting up the Fleet Directory Structure

- Execute `Install-Fleet` on a cluster node (via CredSSP).
- The cmdlet populates the cluster share with files and folders:
  - **Control**: Master scripts orchestrating virtual machines (**`control.ps1, run.ps1`**).
  - **Flag**: Status files orchestrating simultaneous cluster actions (**`go, pause, done`**).
  - **Result**: Performance tracking data (`result.tsv, perfmon.blg`)
  - **Tools**: Binaries and scripts generating storage traffic (**`diskspd.exe, vmfleet.psm1`**)

Task 3: Deploying VMFleet

- Execute `New-Fleet`
- The command will trigger creation of internal VM switches and deploy the number of VMs equal to **number of physical cores** (default).
- Uses:
  - BaseVHD
  - AdminPass
  - ConnectUser / ConnectPass
- All VMs land in paused state.

Optional VM Hardware Tuning (`Set-Fleet`)

- Adjust VM configuration:
  - ProcessorCount
  - MemoryStartupBytes
  - Dynamic vs static memory via MemoryMinimumBytes / MaximumBytes
- Must respect CPU overcommit boundaries.

Starting VMFleet and Running Workloads

- Consider opening two PowerShell terminals:
  - `Watch-FleetCluster` (monitoring)
  - `Start-Fleet` (starts all VMs)
- VMs enter active state and poll for `run.ps1` updates.
- Initiate simulated workloads:
  - Specific workload e.g. `Start-FleetSweep -b 4 -t 8 -o 8 -w 0 -d 300 -p`
  - Pre-defined multi-workloads via `Measure-FleetCoreWorkload`

Understanding the Four Predefined Workloads

- General:
  - 4K reads, queue depth 32, varied write ratios; simulates baseline mixed workload.
- Peak:
  - maximize IOPS (hero number), 4K random-read with small file.
- VDI:
  - sequential write + read cycles, mimicking desktop virtualization I/O.
- SQL:
  - OLTP + log patterns: high random IO + sequential logging writes.

Monitoring Progress (Watch-FleetCluster)

- Displays real-time cluster counters:
  - CSVFS IOPS
  - Read/Write Bandwidth
  - Latency (ms)
- Aggregates per-node and whole-cluster metrics.
- Essential for identifying bottlenecks during test execution.

Extracting Results After Test Completion

- Results stored in **Collect\Result** folder.
- **Look for:**
  - **ZIP archive** containing:
    - **SDDCDiagnosticInfo** logs
    - Per-VM PerfMon outputs
    - XML configurations
  - **result.tsv**: aggregated cluster telemetry
- **Use result.tsv for analysis by using Excel or more advanced analytics tools.**

Interpreting result.tsv

- **Workload** (General, Peak, VDI)
- **RunType** (Baseline, AnchorSearch, Default)
- **VMCount**, **FleetVMPercent**
- **ProcessorCount**, **MemoryStartupBytes**
- **VMAlignmentPct** (70% or 100%)
- **Performance metrics:**
  - IOPS, AverageCPU
  - AverageCSVReadBW / WriteBW
  - AverageCSVReadIOPS / WriteIOPS
  - **Latency percentiles**: ReadMilliseconds50/90/99 and WriteMilliseconds50/90/99

Practical Result Analysis

- **Does performance scale** with VM count?
- **Latency impact** under Peak and VDI loads.
- How **alignment (70% vs 100%)** influences read/write behavior.
- **Compare results against:**
  - Vendor expectations
  - Previous test runs
  - Hardware baselines

Cleanup

- Use **Remove-Fleet** to delete all VMs and virtual switches.
- Delete CSV volumes created for VMFleet.
- Disable **CredSSP** after automation steps.


opsgility